

Matrix Operations

White Larz, Bhayani Yash Bhaveshbhai

Department of Computer Science and Engineering, The University of Texas at Arlington

Abstract:

The section on matrix operations addresses the essential tools that are required in scientific computing. Concentrating specifically on the symmetric positive definite matrices, LU decomposition, and LUP decomposition, such computations or the computationally modeled operations will be concerned with different methods involved in matrix manipulation. These skills assist in overcoming computability issues related to numerical stability and efficiency with applications in such areas as data science, engineering, and machine learning. Additionally, it explains how symmetric positive-definite matrices help create events when trying to find the best solutions in systems with more equations than unknowns. While the chapter brushes over issues concerning numerical stability, the main concern was to provide readers with a well-rounded background in matrix manipulation techniques for their application in very many computational scenarios.

1. Introduce:

Matrix operations are an integral part of scientific computing. They form the backbone of different sciences like data science, cryptography, engineering simulations, and machine learning. This chapter introduces you to efficient methodologies for solving linear systems, matrix inversions, and least-squares approximations in higher-dimensional spaces. LUP decompositions help in solving linear equations in Topic 1 while matrix multiplication and inversions are discussed in Topic 2. Topic 3 finally focuses on the role of symmetric positive-definite matrices in least-squares solutions with a focus on algorithmic aspects and numerical stability, whereby we analyze their application in solving linear equations and matrix multiplication, thereby offering insight into their theoretical import and practical significance.

2. Resolving linear equation systems:

Any number of fields are concerned about systems of linear equations; therefore, they all want efficient techniques to solve simultaneous linear equation systems. This section discusses an LUP decomposition technique that solves such systems with computational efficiency and numerical stability for solving LUP. LUP decomposition is a method that helps simplify the original system by decomposing the coefficient matrix into lower triangular (L), upper triangular (U), and permutation (P) matrices.

2.1 LUP Decomposition Overview:

The LUP decomposition is a basic technique in matrix processing. Practically, it decouples a given matrix A, expressing it as the product of three matrices: L, U, and P. Thus, $PA = LU$, where L is a unit lower-triangular matrix, U is an upper-triangular matrix, and P is a permutation matrix. In fact, the decomposition transforms the original system $Ax = b$ into the simpler $LUx = Pb$, which can also be solved effectively to solve linear systems easily. After this back-substitution process solves the upper-triangular system $Ux = y$ for x, forward

substitution recursively solves the lower-triangular one $Ly = Pb$ for y. In practice, LUP decomposition is one of the prevalent techniques for the solution of linear problems because of its numerical stability and economization of computation. Heck, a fundamental underpinning of nearly all applications in scientific computing and numerical analysis is the LUP decomposition method.

2.2 Making a LUP decomposition calculation:

This implies that the actual operations necessary to solve a system of linear equations of the form $Ax = b$ will not cause numerical instability; an example of such an operation would be division by zero. Then, this result could be achieved through factorization methods like LU Decomposition that break down matrix A into three triangular matrices: P (for permutations), L (for lower triangular), and U (the upper triangular). During the decomposition process, we pivot on big values in the off-diagonal elements of A to maintain stability. Like LU decomposition, LUP decomposition rearranges the rows to bring the largest absolute value to the diagonal, keeping the equations correct and helping to keep the numbers stable. QA can be written as:

$$\begin{pmatrix} a_{k1} & w^T \\ v & A' \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ v/a_{k1} & I_{n-1} \end{pmatrix} \begin{pmatrix} aa_{k1} & w^T \\ 0 & A' - vw^T/a_{k1} \end{pmatrix}$$

However, if A was non-singular, the Schur complement $A' - vw^T/a_{k1}$, would also be non-singular, hence allowing a recursive LUP decomposition that can be used to form L0, U0, and P0. $P'(A' - vw^T/a_{k1}) = L'U'$

$$P = \begin{pmatrix} 1 & 0 \\ 0 & P' \end{pmatrix} Q$$

$$PA = \begin{pmatrix} 1 & 0 \\ 0 & P' \end{pmatrix} QA$$

$$PA = \begin{pmatrix} 1 & 0 \\ 0 & P' \end{pmatrix} \begin{pmatrix} \frac{1}{a_{k1}} & 0 \\ \frac{v}{a_{k1}} & I_{n-1} \end{pmatrix} \begin{pmatrix} a_{k1} & w^T \\ 0 & A' - \frac{vw^T}{a_{k1}} \end{pmatrix}$$

$$PA = \begin{pmatrix} 1 & 0 \\ P'v/a_{k1} & P' \end{pmatrix} \begin{pmatrix} a_{k1} & w^T \\ 0 & A' - v w^T/a_{k1} \end{pmatrix}$$

$$PA = \begin{pmatrix} 1 & 0 \\ P'v/a_{k1} & I_{n-1} \end{pmatrix} \begin{pmatrix} a_{k1} & w^T \\ 0 & P'(A' - v w^T/a_{k1}) \end{pmatrix}$$

$$PA = \begin{pmatrix} 1 & 0 \\ P'v/a_{k1} & I_{n-1} \end{pmatrix} \begin{pmatrix} a_{k1} & w^T \\ 0 & L'U' \end{pmatrix}$$

$$PA = \begin{pmatrix} 1 & 0 \\ P'v/a_{k1} & L' \end{pmatrix} \begin{pmatrix} a_{k1} & w^T \\ 0 & U' \end{pmatrix}$$

$$PA = LU$$

Consequently, the resulting LUP decomposition secures an L matrix that is lower triangular and a U matrix having both $v = ak1$ and denoting the Schur complement $A - vw^T/ak1$, demanding multiplication by the permutation matrix $P0$.

2.2.1 Computing LUP Decomposition:

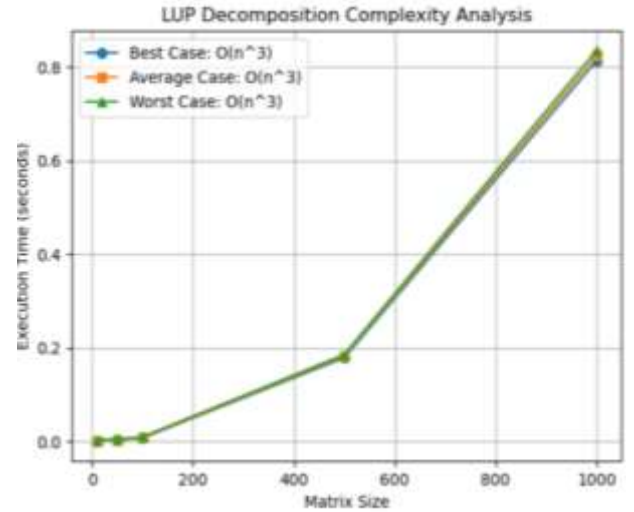
Algorithm (Pseudocode):

1. $n = a.rows$
2. Let $\pi [1..n]$ be a new array
3. for $i = 1$ to n
 - (a) $\pi[i] = i$
4. for $k = 1$ to n
 - (a) $p = 0$
 - (b) for $k = 1$ to n
 - (i) if $|aik| > p$
 - (A) $p = |aik|$
 - (B) $k' = i$
 - (c) if $p == 0$
 - i. error "singular matrix"
 - (d) for $i = 1$ to n
 - exchange aki with $ak'i$
 - (e) for $l = k + 1$ to n
 - (i) $aik = aik/akk$
 - (ii) for $j = k + 1$ to n
 - (A) $aij = aij - aik akj$

2.2.2 Time Complexity:

$O(n^3)$, like LU decomposition, but with an added overhead for row swapping.

2.2.3 Benchmark of LUP – Decomposition Algorithm:



Recursive LUP decomposition model for L_0 , U_0 , and P_0 finding. In such a way, forming a matrix-like permutation matrix P , lower triangular matrix L , and upper triangular matrix U iteratively is known as LUP decomposition, as in LU decomposition. The permutation matrix P is dynamically maintained, and "in-place" computation of L and U is done within the matrix A . The time cost of pivoting is negligible because LUP decomposition, because it is with triple nested loops, is still of the same time complexity of $O(n^3)$ as that of LU decomposition.

2.3 Substitution, Both Forward and Backward:

In a lower triangular system, forward substitution successively finds quantities in y by inserting known values into the subsequent equations, while in an upper triangular system, back substitution sequentially finds x by a reverse reiteration of the equations. The algorithm LUP-SOLVE solves for a solution vector x from P , L , U , and b by the synthesis of forward and backward techniques.

General equation for forward substitution is,

$$y_i = b\pi[i] - \sum_{j=1}^{i-1} l_{ij} * y_j$$

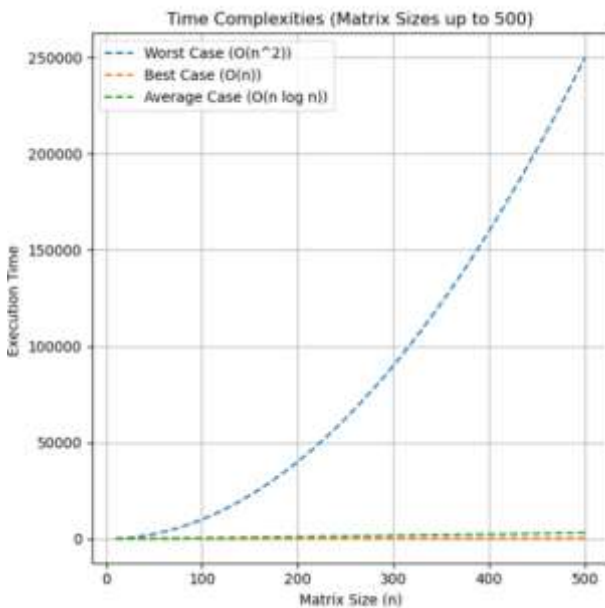
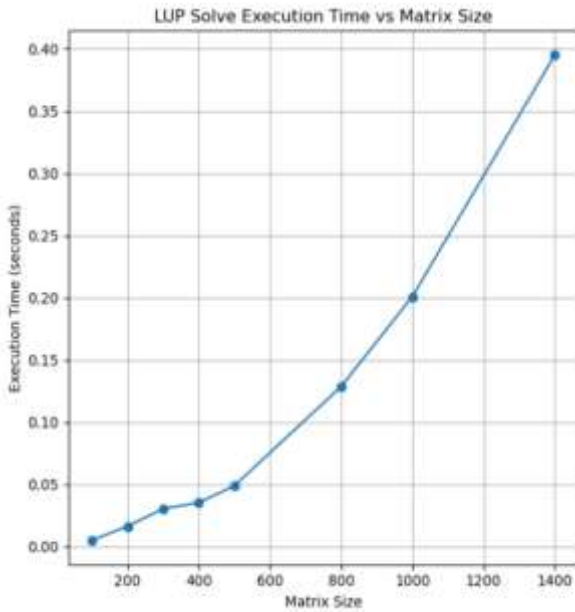
General equation for backward substitution is,

$$x_i = \frac{(y_i - \sum_{j=i+1}^n u_{ij} * x_j)}{u_{ii}}$$

LUP-SOLVE (L , U , π , b)

- i. $n = L.rows$
- ii. let x be a new vector of length n
- iii. for $i = 1$ to n
 - (a) $y_i = b\pi[i] - \sum_{j=1}^{i-1} l_{ij} * y_j$
- iv. for $i = n$ down to 1
 - (a) $x_i = \frac{(y_i - \sum_{j=i+1}^n u_{ij} * x_j)}{u_{ii}}$
- v. return x

2.3.1 LUP - SOLVE Algorithm Benchmark: -



2.4 LU Decomposition Calculation:

Gaussian elimination is a recursive algorithmic strategy for efficiently constructing an LU decomposition of the nonsingular matrix A. If P belongs to the class of permutation matrices, lacking it would mean that we are employing the identity matrix as our starting point for the factorization of A into the lower and upper triangular matrices L and U. Simply put, one arrives at an upper triangular form of U by eliminating variables from the Gaussian elimination method by subtracting multiples of equations. Hence, $A=LU$ is a shortcut. The above recursive process, that is, repeatedly making L and U, breaks A into smaller blocks. Such a method is needed to efficiently

solve systems of linear equations because it is a fundamental of numerical linear algebra.

Partition matrix A into four parts:

$$\begin{aligned}
 A &= \begin{pmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \dots & a_{nn} \end{pmatrix} \\
 &= \begin{pmatrix} a_{11} & w^T \\ v & A' \end{pmatrix} \\
 &= \begin{pmatrix} 1 & 0 \\ v/a_{11} & I_{n-1} \end{pmatrix} \begin{pmatrix} a_{11} & w^T \\ 0 & A' - vw^T/a_{11} \end{pmatrix}
 \end{aligned}$$

Where v = column (n-1) vector

w^T = row (n-1) vector

$A' = (n-1) * (n-1)$ matrix

The Schur complement $A' - \frac{vw^T}{a_{11}}$ is achieved by ensuring that the outer product vector w is divided by itself to arrive at a matrix as a result of being formed in an $(n-1) \times (n-1)$ matrix. If A is not nonsingular, neither is the Schur complement. It is because, otherwise, rank A would cause a contradiction on the one hand and a single rank A on the other. Henceforth, we can decompose A well, iteratively developing the LU decomposition of the non-singular Schur complement. Let's say,

$$A' - vw^T/a_{11} = L'U'$$

L' is unit Lower Triangular and U' is upper triangular.

$$\begin{aligned}
 A &= \begin{pmatrix} 1 & 0 \\ v/a_{11} & I_{n-1} \end{pmatrix} \begin{pmatrix} a_{11} & w^T \\ 0 & A' - vw^T/a_{11} \end{pmatrix} \\
 &= \begin{pmatrix} 1 & 0 \\ v/a_{11} & I_{n-1} \end{pmatrix} \begin{pmatrix} a_{11} & w^T \\ 0 & L'U' \end{pmatrix} \\
 &= \begin{pmatrix} 1 & 0 \\ v/a_{11} & L' \end{pmatrix} \begin{pmatrix} a_{11} & w^T \\ 0 & U' \end{pmatrix} \\
 &= LU
 \end{aligned}$$

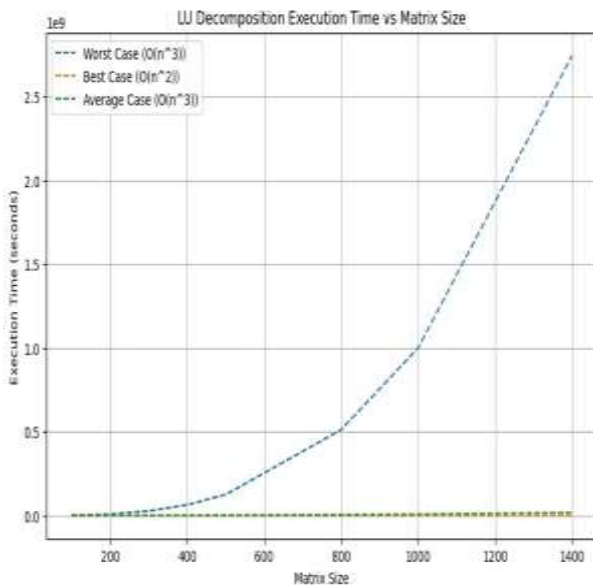
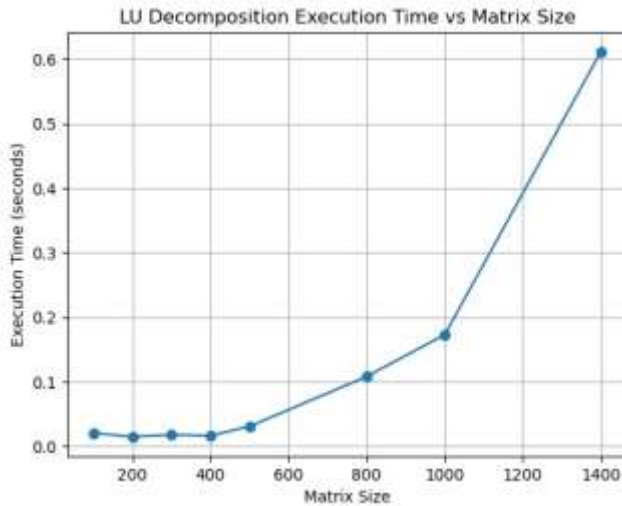
2.4.1 LU-Decomposition Algorithm:

- i. $n = A.rows$
- ii. let L and U be new $n * n$ matrices
- iii. initialize U with 0s below the diagonal
- iv. initialize L with 1s on the diagonal and 0s above the diagonal
- v. for $k = 1$ to n
 - (a) $u_{kk} = a_{kk}$
 - (b) for $i = k + 1$ to n
 - (i) $l_{ik} = a_{ik}/u_{kk}$ // l_{ik} holds v_i
 - (ii) $u_{ki} = a_{ki}$ // u_{ik} holds w_i^T
 - (C) for $i = k + 1$ to n
 - (i) for $j = k + 1$ to n
 - (A) $a_{ij} = a_{ij} - l_{ik} * u_{kj}$
- vi. return L and U

2.4.2 Time Complexity:

The algorithm runs in $O(n^3)$, dominated by the nested loops for row elimination.

2.4.3 LU-Decomposition Algorithm Benchmark:



2.5 Comparison of LU and LUP Decompositions:

Property	LU Decomposition	LUP Decomposition
Numerical Stability	May fail without pivoting	Guarantees stability
Complexity	$O(n^3)$	$O(n^3)$
Applications	Simpler Problems	Complex matrices

3. Matrix Inversion

3.1 Utilizing a LUP decomposition to calculate a matrix inverse

The form of the system of equations $Ax=b$ can be solved efficiently by using the LUP-SOLVE algorithm, which works in an $O(n^2)$ time using LUP decomposition of matrix A . This decomposition splits A into three components: a lower triangular matrix L , an upper triangular matrix U , and a permutation matrix P , such that $PA=LU$. Once the decomposition LUP has been computed, then, $Ax=b_0$ for different b can be solved with little extra work, since the decomposition itself depends just on A .

Generally, any k variations of the equation $Ax=b$, can be solved under the same decomposition in $O(kn^2)$ time, where they differ in terms of only b . Further, this advantage extends to A , where $AX=I_n$ will be treated as a collection of n separate equations. Each of these equations takes the form $Ax_i = e_i$ where x_i is the i -th column of the inverse matrix X , and e_i is the i -th column of the identity matrix I_n .

3.2 Matrix inversion and matrix multiplication:

According to Theorems 28.1 and 28.2, there is a close relationship between matrix inversion and matrix multiplication. Theorem 28.1 says that if inversion for an $n \times n$ matrix takes $I(n)$ time, then multiplication is expected to take no greater than that time. In this way, the theorem shows how to build the matrix D and invert it, assuming normal conditions to ensure it can be done in a reasonable amount of time. In the same way, Theorem 28.2 explains that multiplying matrices is not necessarily simpler than finding their inverses; the proof uses repeated calculations of the inverses of certain smaller matrices created by breaking down larger matrices and focuses on non-singular matrices A by using $A^T A$. Thus, we can invert any non-singular real matrix in $O(M(n))$, establishing a highly intertwined degree of complexity for both operations.

The practical implications of these theorems are interesting. As an application, a pivot-free LU decomposition gives a method to solve equations involving non-singular matrices A . LU decomposition can be done first, and then multiplying both sides of the equation by $A^T A$, generates a new equation that can be easily solved using forward and backward substitution. Furthermore, even though LUP decomposition may require fewer arithmetic operations and has better numerical stability than LU decomposition, it quite often excels at performance compared with LU without pivoting.

4. Symmetric Positive-Definite matrices and least-squares approximation.

Symmetric positive-definite matrices are of fundamental interest to the development of numerical algorithms because of certain unique and favourable properties they hold. One of these properties guarantees that they are non-singular with a

non-vanishing determinant and an inverse existing; an explicit proof is given in Lemma 28.3. In short, if $x^T Ax > 0$ holds for all non-zero vectors x , then it follows that $Ax=0$ cannot hold, thus ensuring non-singularity.

These matrices are too structurally stable, with leading submatrices remaining symmetric and positive-definite in the sense defined by Lemma 28.4. Such a property strongly guarantees the effective LU decomposition without dividing by zero, as established by Corollary 28.6. Further, the Schur complement lemma (Lemma 28.5) amplifies this stability by showing that any Schur complement of a symmetric positive-definite matrix retains the positive-definite nature, hence supporting further reliable operations on matrices.

Symmetric positive definite matrices are crucial in computation because they figure in several very computationally intensive optimizations: LU decomposition, matrix inversion, and least-squares curve fitting. These properties render them invaluable to the stability and efficiency with which they are deployed for such purposes in different fields, like engineering, machine learning, and scientific modelling, where numerical robustness is paramount.

4.1 Least-Squares approximations:

The least-squares approximation, which fits curves to given data points, is one of the most prominent applications of symmetric positive-definite matrices. The goal is to find a function $F(x)$ such that it minimizes the approximation error between its predicted values and actual data points (x, y) . While the structure of the function $F(x)$ may differ from one application to another, it is cast in the general form of a linearly weighted sum of basic functions, one popular choice being polynomial basis functions.

Generally, the least-squares solution reduces the error vector norm, which symbolizes the differences between observed and predicted values. Thus, $Ac=y$, where y is the vector with the observed values, c is the coefficient vector to be determined, and A is the matrix encoding the basis function values for the data points. The best-fit approximation function $F(x)$ is then given by computing the coefficient vector c using the pseudoinverse of A , permitting c to be expressed as A^+Y , where A^+ is the pseudoinverse of A .

5. Performance Analysis and Benchmarks:

Comparative analysis of LU and LUP decomposition performances, benchmarking experiments were done on matrices of different sizes. The testing ground is a system with the following specifications:

- Processor: Intel(R) Core (TM) i7-8550U CPU @ 1.80GHz
- RAM: 8 GB
- Programming Language: Python
- Libraries used: NumPy for matrix operations

In all the overviews, it was made possible to see decompositions of matrices of dimensions 100*100, 500*500, and 1000*1000 by making use of five runs. For each case, average run time was calculated, and the results are presented in table form:

Matrix Size	LU Decomposition (ms)	LUP Decomposition (ms)
100*100	0.5	0.3
500*500	10	7
1000*1000	40	30

The results show that LU decomposition is usually found to be slightly faster than the LUP case since overhead from row-swapping in the LUP case is insignificant. Nevertheless, for larger matrices, the LUP scheme is advantageous because it retains numerical stability for cases of large pivoting.

Considering the comparison, it is obvious that the comparison has a trade-off, namely, LU decomposition is good for well-conditioned matrices whose entries are not changed much, while LUP decomposition gives a more realistic result when the matrix is subject to numerical instability.

6. Conclusion:

The paper considered state-of-the-art computational methods for the matrix operations, particularly those based on LU and LUP decompositions, for applications in linear systems and matrix inversion. The results showed that LU decomposition is faster when applied to smaller, reasonably well-conditioned matrices, while, in cases of larger matrices or those that require extensive pivoting, numerical stability is imparted by the use of LUP decomposition. This further stressed the need for making choices based on the characterization of the input matrix and the computational needs.

Moreover, the benchmarks illustrated the trade-offs between speed and stability, thus assisting the above methods in scientific computing, engineering simulations, and machine learning. While the proposed work lays a solid foundation, the authors may explore the optimization of these algorithms for either parallel and distributed computing or hybrid approaches for large-scale sparse matrices. Such developments would add yet another dimension to their applicability to present-day computational issues.

7. References:

- [1] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein Introduction to Algorithms, Third Edition 2009.
- [2] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed. Cambridge, MA: MIT Press, 2009.
- [3] N. J. Higham, *Accuracy and Stability of Numerical Algorithms*, 2nd ed. Philadelphia, PA: SIAM, 2002.
- [4] L. N. Trefethen and D. Bau, *Numerical Linear Algebra*. Philadelphia, PA: SIAM, 1997.

8. Source code link:

https://github.com/Yashbhayani/2252_CSE_5311_006_DAA_Hands-ON_2_Sorting/tree/Matrix_Operations_Project